

VulnCorp Helpdesk Lab Practices

20 Bug Bounty Lab Practices — VulnCorp Helpdesk on Metasploitable2

A full-workflow lab manual: recon → mapping → exploitation → chaining → reporting, using VulnCorp Helpdesk deployed on Metasploitable2 (or via `setup-modern.sh` / Docker) as the live target. Each lab names its required tools, the difficulty tier to set, numbered steps, a success check, and a “Report it” note showing what that finding would look like in a real submission — so by Lab 20 you’re not just exploiting bugs, you’re practicing the full bounty-hunter workflow.

Before Lab 1: confirm the app is deployed and reachable (`README.md` section 1 or 2), and you know its IP/URL. Set difficulty via **Admin Panel** → **Difficulty Settings** as each lab specifies. Seeded accounts: `admin/admin123`, `sam/support123` (support), `alice/alice123`, `bob/bob123` (all four log in in a single step), and `carol/carol123` — the one account with 2FA enabled, needed for Lab 29 specifically.

Phase A — Recon & Mapping (Labs 1–4)

Lab 1: Full Port & Service Enumeration

Tools: nmap **Tier:** any **Objective:** Build a complete picture of what’s running on the target before touching the web app at all.

Steps:

1. Ping-sweep the subnet to confirm the target is up: `nmap -sn <subnet>/24`
2. Run a full TCP port scan with version detection: `nmap -sV -p- <target-ip>`

3. Record every open port and its reported service/version in your notes (use the Recon Note Template from the Toolkit).
4. Cross-reference the results against known service versions — note anything with a publicly known CVE (e.g. vsftpd 2.3.4 on Metasploitable2, PHP version, Apache version).
5. Identify which port serves VulnCorp Helpdesk specifically.

Success check: You have a documented list of every open port, its service, and version, with the VulnCorp Helpdesk port identified.

Report it: Not a finding on its own, but this becomes your report's "**Scope and Methodology**" section — real reports document what was tested, not just what was found.

Lab 2: Web Stack Fingerprinting

Tools: whatweb, curl, browser DevTools **Tier:** simple **Objective:** Identify the exact technology stack serving the app without being told.

Steps:

1. Run `whatweb http://<target>/vulnapp/` and record what it reports for server, language, and framework.
2. Manually check response headers: `curl -I http://<target>/vulnapp/index.php` — note `Server` and any `X-Powered-By`.
3. Trigger a deliberate error (e.g. submit a malformed request) and see if any stack trace or version info leaks.
4. Compare your findings to what section 1 of the README documents about the target stack — did you independently arrive at the same answer?

Success check: You can state the exact web server, PHP version (or version range), and database in use, backed by evidence you collected yourself.

Report it: If a verbose version header or stack trace leaked, that's a **Low/Informational** finding — "Server version disclosure via HTTP headers." Note it for your final report even though it's not the main finding.

Lab 3: Content & Endpoint Discovery

Tools: ffuf or gobuster or dirb, SecLists wordlists **Tier:** hard (this is where the payoff endpoint exists) **Objective:** Discover an endpoint that isn't linked from anywhere in the UI.

Steps:

1. Set difficulty to **hard** via Admin Panel.
2. Run `ffuf -u http://<target>/vulnapp/user/FUZZ.php -w /usr/share/seclists/Discovery/Web-Content/common.txt -mc 200,301,302,403`
3. Also fuzz the `admin/`, `support/`, and root directories separately.
4. For every hit, manually visit it and note what it does, what HTTP methods it accepts, and whether it enforces authentication.
5. Compare your discovered endpoint list against what's actually linked in the dashboard navigation — what did you find that isn't linked anywhere?

Success check: You've discovered `profile_export.php` (or an equivalent hidden endpoint) purely through brute-forcing, before reading any hint.

Report it: This discovery technique — “endpoint enumeration via content discovery” — belongs in your methodology section, and the endpoint itself likely feeds into Lab 12's finding.

Lab 4: Full Application Mapping with Burp Suite

Tools: Burp Suite (Proxy + Site Map) **Tier:** any **Objective:** Build a complete map of the app's pages, forms, parameters, and role boundaries.

Steps:

1. Configure your browser to proxy through Burp (127.0.0.1:8080).
2. Log in and click through the app as each of the four seeded accounts in turn (admin, sam, alice, bob).
3. In Burp's Site Map, review every distinct URL, form, and parameter discovered.

4. Note which pages return a 403 for which roles — this tells you where access control is *supposed* to apply.
5. Identify the session mechanism (cookie name, whether it's `HttpOnly` / `Secure`) and where the difficulty-tier setting lives (hint: it's server-side, not in a cookie or hidden field).

Success check: A written map (screenshot or list) covering all four roles' visible pages, plus your notes on where role-based restrictions appear to exist.

Report it: This map becomes the backbone of your final report's scope description, and directly informs which endpoints you'll test for access-control bugs in Labs 8, 12, and 17.

Phase B — Simple Tier: Finding the Obvious Bugs (Labs 5–9)

Lab 5: SQL Injection Authentication Bypass

Tools: Browser or Burp Repeater **Tier:** simple **Objective:** Log into the app as `admin` without knowing the password.

Steps:

1. Set difficulty to **simple**.
2. On the login form, enter a single quote (`'`) in the username field and submit — observe the error behavior.
3. Reason about the underlying query structure based on the error.
4. Craft a payload that closes the string early and neutralizes the rest of the query: try `admin' --` as the username, any value as password.
5. Confirm you're logged in as `admin` without ever knowing `admin123` .

Success check: You're on the admin dashboard having never entered the real password.

Report it: Critical. Title: *"SQL Injection in Login Form Allows Full Authentication Bypass."* Include the exact payload, a screenshot of the resulting authenticated session, and note the business impact:

complete compromise of any account, including admin, with zero prior credentials.

Lab 6: Data Extraction via UNION-Based SQLi

Tools: Burp Suite, sqlmap **Tier:** simple **Objective:** Extract every username and password hash from the database through the login form alone.

Steps:

1. With difficulty still on **simple**, submit the login form once with a **real seeded username** (e.g. `admin`) and a wrong password — not a made-up username. Capture that exact POST request in Burp and save it as a `.req` file.
2. Run `sqlmap -r login.req -p username --suffix="-- -" --drop-set-cookie --dump` and let it identify and exploit the injection automatically. The two extra flags matter here: `--suffix` makes every payload sqlmap tries end in a SQL comment (the same technique as Lab 5's bypass), and `--drop-set-cookie` stops sqlmap from reusing a session a successful payload just logged in — without both, automated detection against this specific login form can fail or stall partway through extraction even though the injection is real.
3. Separately, manually determine the column count via `UNION SELECT` trial and error, and craft a manual payload that reflects data back into the page.
4. Compare what sqlmap found against your manual extraction.

Success check: A full dump of the `users` table, including password hashes, obtained via at least one method.

Report it: Same finding as Lab 5, but now with concrete evidence of data exposure — attach the extracted table (redact real credentials if this were a live program) as your Proof of Concept. This upgrades the report's **Impact** section from “could log in as anyone” to “extracted the entire user database.”

Lab 7: Stored XSS via Support Tickets

Tools: Browser DevTools **Tier:** simple **Objective:** Get JavaScript to execute in another user's browser via a submitted support ticket.

Steps:

1. Log in as `alice` . Submit a ticket with `<script>alert(document.cookie)</script>` in the message field.
2. View "My Tickets" and confirm the script executes.
3. Think through the chain: support agents (`sam`) read every ticket in the queue. Log in as `sam` and view the Support Queue — does the same script fire there too?
4. Consider (don't need to build) what a real payload would do instead of `alert()` — e.g., exfiltrate the session cookie to an attacker-controlled listener.

Success check: Confirmed script execution both in your own ticket view and in the support agent's queue view.

Report it: Medium-High, depending on who it reaches. Title: *"Stored XSS in Ticket Submission Executes in Support Agent Context."* Note explicitly that this isn't self-XSS — it reaches a *different, higher-privileged* user (the support agent) without any action from them beyond normal job duties.

Lab 8: IDOR — Viewing and Editing Other Users' Profiles

Tools: Browser **Tier:** simple **Objective:** View and modify another user's profile data using only your own low-privilege session.

Steps:

1. Log in as `alice` . Note your own profile URL and its `id` parameter.
2. Change the `id` value to another user's ID (try 1 through 4) and reload.
3. Confirm you can see full profile details for accounts that aren't yours — including the password hash field, visible at this tier.
4. Attempt to *edit* another user's profile (not just view) via the same page.

Success check: Full read and write access to at least one other account's profile data.

Report it: High. Title: *"Insecure Direct Object Reference in User Profile Allows Unauthorized Data Access and Modification."* Note both the read (info disclosure) and write (data integrity) impact separately — they're two distinct consequences of the same root cause.

Lab 9: Command Injection via Admin Diagnostics

Tools: Browser, Burp Suite **Tier:** simple **Objective:** Execute an arbitrary OS command on the server via the network diagnostics tool (admin access required).

Steps:

1. Log in as `admin` , navigate to the Network Diagnostics tool.
2. Submit a normal hostname (e.g. `127.0.0.1`) and confirm the ping output renders.
3. Append a shell metacharacter and a harmless command:
`127.0.0.1; id`
4. Confirm the output includes the result of `id` , not just the ping.
5. Try a second, different separator (`&&`) to confirm the vulnerability isn't specific to one character.

Success check: Arbitrary command output (e.g. `uid=0(root)`) or similar) appearing in the tool's response.

Report it: Critical. Title: *"OS Command Injection in Network Diagnostics Tool Allows Remote Code Execution."* This is your strongest finding so far — full RCE. Include the exact payload and full output as PoC.

Phase C — Intermediate Tier: Filter Evasion (Labs 10–12)

Lab 10: Bypassing the SQLi Keyword Filter

Tools: Burp Suite **Tier:** intermediate **Objective:** The app now strips `union` , `select` , `--` , `#` , `;` (lowercase only). Bypass login anyway.

Steps:

1. Set difficulty to **intermediate**. Retry Lab 5's exact payload — confirm it now fails.
2. Test whether the filter is case-sensitive by submitting a single blocked keyword in mixed case.
3. Recall that the original auth-bypass payload doesn't strictly need any of the blocked keywords — only an always-true condition and a comment. Reconstruct a bypass using only unfiltered characters.
4. Document exactly why your new payload evades the specific filter logic (case-sensitivity, or keyword-independence).

Success check: Authenticated as admin again, this time past an active filter.

Report it: Update your Lab 5 report to note: *"The vendor's proposed mitigation (keyword blacklist) is insufficient — see attached bypass."*

This is a realistic real-world scenario: reporting that a partial fix doesn't actually close the finding.

Lab 11: XSS Filter Bypass

Tools: Browser DevTools, PortSwigger XSS cheat sheet (public reference) **Tier:** intermediate **Objective:** The app now strips `<script>` tags case-insensitively. Get JavaScript to execute anyway.

Steps:

1. Confirm your Lab 7 payload is now neutered.
2. Recall that a browser executes JavaScript from many contexts beyond `<script>` tags — event handler attributes fire on their triggering event regardless of tag name.
3. Submit a ticket containing `<img src=x onerror=alert(1)>` and confirm execution.
4. Try at least one more vector (e.g. an `<svg onload=...>` payload) to confirm the filter's blind spot is systemic, not a one-off.

Success check: Confirmed script execution using a payload containing no literal `<script>` substring.

Report it: Same update pattern as Lab 10 — note in your report that tag-name-based blacklisting doesn't address the underlying lack of output encoding.

Lab 12: File Upload Filter Bypass (Content-Type Spoofing)

Tools: Burp Suite (Repeater) **Tier:** intermediate **Objective:** The upload feature now checks the file's Content-Type header. Bypass it to upload a PHP file.

Steps:

1. Attempt to upload a `.php` file normally through the Avatar Upload feature — confirm it's rejected.
2. Intercept the upload request in Burp before it reaches the server.
3. Edit only the `Content-Type` field in the multipart body for your file part to `image/jpeg` (or similar), leaving the actual file bytes and filename untouched.
4. Forward the request and confirm the upload now succeeds.
5. Browse directly to the uploaded file's URL and confirm your PHP code executes.

Success check: A `.php` file uploaded and executing, despite the Content-Type check.

Report it: Critical (same class as any unrestricted-upload RCE). Note specifically in your report *which* control was bypassed and *how* — “validation trusts a client-supplied header instead of inspecting file content” is the root-cause line a developer needs to fix it correctly.

Phase D — Hard Tier: Chained & Secondary Flaws (Labs 13–16)

Lab 13: Cookie-Based SQL Injection via “Remember Me”

Tools: Burp Suite **Tier:** hard **Objective:** The login form itself is now fully parameterized. Find the SQLi that moved elsewhere in the auth flow.

Steps:

1. Set difficulty to **hard**. Confirm the login form itself resists every prior SQLi payload.
2. Log in normally with “Remember me” checked; inspect the resulting cookie.
3. Reason about where else in the app a stored value like this might get read back into a query — a remember-me feature has to look up the token somewhere.
4. Log out, then manually set a crafted `remember_token` cookie value designed to manipulate that lookup query, and reload the login page.
5. Confirm you’re auto-logged-in as an arbitrary account without a username/password at all.

Success check: Authenticated session established purely via a forged cookie value.

Report it: Critical. Title: “SQL Injection via Forgeable ‘Remember Me’ Cookie Bypasses Authentication Entirely.” Emphasize in your report that fixing the *primary* injection point doesn’t guarantee the vulnerability class is gone from the codebase — this is a strong point to make to a client who thinks one fix “solved SQLi.”

Lab 14: Broken Access Control — The Forgotten Endpoint

Tools: Burp Suite, ffuf (from Lab 3) **Tier:** hard **Objective:** Direct profile access is now correctly restricted to your own account. Find where that fix wasn’t applied.

Steps:

1. Confirm Lab 8’s IDOR now correctly 403s at this tier.
2. Revisit your Lab 3 endpoint-discovery notes, or re-run the fuzzing if you skipped it — look specifically for an “export,” “api,” or “download” style endpoint.
3. Test the same IDOR technique (changing the target ID) against that endpoint instead of the main profile page.
4. Confirm it returns full profile data for arbitrary user IDs, unauthenticated by role.

Success check: Data exposure confirmed on the secondary endpoint even though the primary one is fixed.

Report it: High. Title: *“Broken Access Control on /user/profile_export.php Bypasses Fixed IDOR Protection.”* This is a great example of a **defense-in-depth** finding for your report: the fix was applied to one code path but not a functionally-identical one — flag this as a systemic code-review gap, not just a single bug.

Lab 15: Polyglot File Upload for RCE

Tools: exiftool or a hex editor **Tier:** hard **Objective:** The app now validates real image structure via `getimagesize()`. Get PHP execution anyway.

Steps:

1. Confirm a plain `.php` file is now rejected by the image-structure check.
2. Build a polyglot: prepend the 6-byte GIF header (`GIF89a`) to a one-line PHP payload (`<?php echo "pwned"; ?>`).
3. Verify locally that `getimagesize()` (or an online equivalent check) accepts this file as a valid GIF.
4. Upload the file with a `.php` extension via the Avatar Upload feature.
5. Browse to the uploaded file and confirm the PHP portion executes (look for your echoed string in the output, not just the raw file).

Success check: Confirmed PHP execution from a file that also independently validates as a real image.

Report it: Critical. Title: *“Image-Validation Bypass via Polyglot File Enables Remote Code Execution.”* Be precise in your report about *why* this works: `getimagesize()` confirms a file *starts with* valid image data, not that it contains *only* image data — a subtle but important distinction for the fix recommendation.

Lab 16: Race Condition — Multi-Claiming a One-Time Bonus

Tools: Burp Suite Intruder (concurrent mode) or a short async script (Python + aiohttp) **Tier:** simple first, then intermediate **Objective:** Claim

a one-time “welcome bonus” more than once by exploiting a check-then-write timing gap.

Steps:

1. Set difficulty to **simple**. Claim the bonus once normally; confirm your balance is 100 credits.
2. Reset your account (or use a fresh one), then fire 10-15 genuinely concurrent requests at the claim endpoint (multiple browser tabs clicking at once, or several parallel `curl` commands launched together).
3. Check your final balance — more than 100 confirms multiple successful claims.
4. Switch to **intermediate** and repeat — note that a handful of manually-fired requests now mostly fails, because the timing window is much narrower.
5. Use a tool built for real concurrency (Burp Intruder’s concurrent request mode, or a short async script) to land the intermediate-tier race reliably.

Success check: A final balance greater than 100 credits at both tiers, using different techniques for each.

Report it: High. Title: *“Race Condition in Bonus Claim Allows Unlimited Multi-Claiming.”* Note in your report which tooling was required — this matters for severity discussions, since “exploitable with a browser” and “exploitable only with specialized tooling” are meaningfully different risk profiles even for the same root cause.

Phase E — Expert Tier: Logic Flaws & Confirming Fixes (Labs 17–18)

Lab 17: Privilege Escalation via Mass Assignment

Tools: Burp Suite (Repeater) **Tier:** expert **Objective:** The profile-edit form shown to regular users has no role field. Become admin anyway.

Steps:

1. Set difficulty to **expert**. Log in as `alice`, submit a normal profile update, and capture the POST request in Burp.

2. Reason about how the server-side handler likely processes the submission — does it only save fields the form displayed, or everything received?
3. Add a parameter the visible form never included: `role=admin`.
4. Resend the modified request and confirm success.
5. Log out and back in as `alice` — confirm your role has actually changed.

Success check: `alice`'s account now has the `admin` role, achieved purely by adding an unexpected POST field.

Report it: Critical. Title: *"Mass Assignment in Profile Update Allows Privilege Escalation to Administrator."* This is a good report to practice writing precisely: the vulnerability isn't in what the UI shows, it's in what the server accepts — make that distinction explicit for the developer reading your report.

Lab 18: Confirming a Fix — CSRF Token Validation

Tools: Browser, a scratch HTML file **Tier:** hard, then expert **Objective:** Build a CSRF proof-of-concept against an unprotected endpoint, then confirm the same PoC correctly fails once a token is required.

Steps:

1. Set difficulty to **hard**. Build a minimal auto-submitting HTML page targeting `/admin/create_user.php` with fields for a new backdoor admin account (`username`, `password`, `role=admin`, etc.).
2. While logged in as `admin` in one browser tab, open your crafted page in another tab.
3. Log out, then log in with the credentials you set — confirm a persistent backdoor admin account now exists, created without the real admin ever intending to.
4. Switch difficulty to **expert** and reload/resubmit the exact same PoC page.
5. Confirm it now fails, and capture the specific error message returned.

Success check: Successful backdoor creation at hard tier; a clean, specific rejection at expert tier.

Report it: This is two report entries in one lab. First, **Critical:** “*CSRF on Admin User Creation Allows Silent Backdoor Account Creation.*” Second, a **remediation verification note:** confirm to the client that the CSRF token fix works as intended, including your re-test evidence — real bounty programs often want confirmation that a patch actually closes the reported issue, not just the original finding.

Phase F — API Testing & Full Report Writing (Labs 19–20)

Lab 19: API Security Testing — Broken Object Level Authorization

Tools: Postman or Burp Suite, curl **Tier:** simple through expert (test all four) **Objective:** Practice testing a JSON API specifically, not just HTML forms, against `/api/tickets.php`.

Steps:

1. Set difficulty to **simple**. With no authentication at all, request `/api/tickets.php` and `/api/tickets.php?id=1` — confirm full, unauthenticated data exposure.
2. Set difficulty to **intermediate**. Log in as `alice` and request another user's ticket by ID — confirm the API requires login but performs no ownership check.
3. Set difficulty to **hard**. Confirm the single-ticket lookup is now correctly scoped, then test the *list* mode (no `id` parameter) — confirm it still leaks every ticket, since the fix wasn't applied to that code path.
4. Set difficulty to **expert**. Confirm both modes are now correctly scoped to the authenticated user.
5. Throughout, use Postman (or Burp) to save each request as a reusable collection — this is standard practice for API testing on real engagements, where you'll re-run the same requests against many parameter values.

Success check: A documented set of requests/responses showing the exact access-control behavior at each of the four tiers.

Report it: High for the simple/intermediate findings, note the hard-tier list-mode gap as its own distinct finding (“**BOLA in List Mode Persists After Single-Object Fix**”) — same lesson as Lab 14, now in an API context. This is a good moment to practice writing a report for an API finding specifically: no screenshots of a UI, just clean request/response pairs as your PoC.

Lab 20: Full Engagement Report — Compiling Labs 1–19

Tools: The Report Writing Template and Severity Cheat Sheet (Toolkit)

Tier: N/A — this is the writing lab **Objective:** Compile your strongest 5–8 findings from Labs 1–19 into a single, professional engagement report, as if VulnCorp Helpdesk were a real program you were submitting to.

Steps:

1. Review your notes from every prior lab and select the 5–8 most significant, well-evidenced findings — prioritize variety (don’t submit five SQLi findings when you also have RCE, IDOR, and a race condition available).
2. For each finding, use the **Report Writing Template**: a specific title, severity (rated via the **Severity Cheat Sheet**, not guessed), exact steps to reproduce, a clean PoC (request/response or screenshot), business-level impact, and a remediation recommendation specific enough for a developer to act on.
3. Write an **Executive Summary** at the top: 3-4 sentences, no jargon, stating the overall risk posture — this is what a non-technical stakeholder reads first.
4. Order your findings by severity, Critical first.
5. For at least one finding, explicitly note a **chain**: e.g., “the stored XSS in Lab 7, combined with the CSRF from Lab 18, could let an unauthenticated attacker gain full admin access with zero valid credentials at any point” — chaining lower-severity bugs into a higher-impact story is a core bounty-hunting skill, not just listing bugs in isolation.
6. Proofread for one thing specifically: could a developer who has never seen this app reproduce every finding using only what you wrote? If not, add detail until they could.

Success check: A complete, submission-ready report covering at least 5 distinct vulnerability classes, correctly severity-rated, with at least one explicit vulnerability chain called out.

Report it: This *is* the report. Treat it as your portfolio piece — this is the artifact you'd actually want to show a real bug bounty program, a hiring manager, or use as a writing sample when applying to programs that ask for one.

Bonus Phase — API Token (JWT) & CTF Capstone (Labs 21–22)

These two labs are newer additions, covering the API Token (JWT) Auth and API mass-assignment modules added after the original 20-lab sequence above. They're placed after the Lab 20 report-writing capstone rather than between Labs 19 and 20 specifically so Lab 20's "compile Labs 1–19" scope stays accurate — treat this phase as an optional extension once the core workflow above feels comfortable, not a prerequisite for it.

Lab 21: API Security Testing — JWT Auth Flaws

Tools: Burp Suite, curl, a scratch script to base64url-encode JSON (or `python3 -c`) **Tier:** simple through expert (test all four) **Objective:** Practice forging and abusing JSON Web Tokens against the API Token (JWT) Auth module (`/api/auth_token.php` , `/api/ticket_update.php`).

Steps:

1. Set difficulty to **simple**. Hand-craft an unsigned token (header `{"typ":"JWT","alg":"none"}` , payload `{"sub":1,"username":"admin","role":"admin"}` , empty signature segment) and use it as a Bearer token against `/api/ticket_update.php` — confirm it's accepted with zero valid credentials.
2. Set difficulty to **intermediate**. Confirm the exact lowercase `none` is now blocked, then retry with `alg: "None"` — confirm the case-sensitive filter still misses it.

3. Set difficulty to **hard**. Get a real token from `/api/auth_token.php`, decode its `exp` claim, and confirm the API still accepts it well past that timestamp — the signature is real, but expiry is never checked.
4. Set difficulty to **expert**. Confirm an expired token is now rejected — then get a fresh token for your own account, change your own password via `/user/change_password.php`, and confirm the *old* token still works anyway (no revocation on credential change).
5. Throughout, keep a copy of each forged/captured token in your notes — a real JWT finding's PoC is the token itself plus the request it unlocks.

Success check: Documented proof of the alg-confusion bypass (simple+intermediate), the missing-expiry replay (hard), and the missing-revocation replay (expert), each with the exact token and request used.

Report it: The simple/intermediate bypasses are **Critical** (full authentication bypass with zero credentials). The hard-tier missing-expiry and expert-tier missing-revocation findings are each their own **Medium-to-High** report — “token lifetime” bugs are a distinct, common finding category in real JWT-based APIs, separate from the forgery itself.

Lab 22: CTF Capstone — Chain the API Flaws for a Flag

Tools: Browser or Burp Suite, curl **Tier:** any (this chain works at every tier) **Objective:** Walk the “The Forgotten Export” chain end-to-end — from a brand-new, unprivileged login to reading an admin-only ticket containing a flag — using bugs you’ve already found earlier in this manual.

Steps:

1. Log in as any seeded non-admin account (e.g. `bob`) and request `/user/profile_export.php?id=1` — this reuses the same forgotten-endpoint IDOR from Lab 14, now leaking an `api_key` field alongside the profile data.
2. Exchange that leaked `api_key` (with username `admin`) for a JWT at `/api/auth_token.php` — this is the legitimate exchange path, not a forged token.

3. Use the resulting token as a Bearer token against `/api/tickets.php?id=3` and recover the flag string in the ticket's message.
4. Separately, forge a completely unsigned token (`alg: none` , `role: admin`) and send it to `/api/ticket_update.php?admin_note=1` — recover the second flag, independent of the app's configured tier.
5. Write up both chains as a single finding: the real value of a CTF-style writeup is showing the *chain*, not just the final step.

Success check: Both flags recovered, with a documented chain for each showing every step from your starting unprivileged login to the flag.

Report it: Write this one as a single **Critical** finding titled something like “*Chained IDOR + Credential Leak + API Auth Allows Full Admin Data Access*” — in a real program, a triager cares far more about a clearly-chained path to high impact than about five separate low-severity tickets for the same underlying root cause.

Bonus Phase — Session & Client-Side Security (Labs 23–26)

These four labs are newer additions, covering the session-fixation, clickjacking, cache-poisoning, and DOM XSS modules added after the original 20-lab sequence and the Labs 21–22 bonus phase above. All four live in features that either never send anything to the server at all (Lab 26) or depend on response headers and session handling rather than payload-style input (Labs 23–25) — a different muscle than the earlier injection-style labs. Run this phase in either order relative to the next one; neither depends on the other.

Lab 23: Session Fixation — Riding In on a Known Session ID

Tools: Browser DevTools (to read and set cookies manually), two browser profiles (or one browser plus curl, to act as “attacker” and “victim” separately) **Tier:** simple through hard **Objective:** Get authenticated as another user by planting a session ID *before* they log in, rather than stealing anything after the fact.

Steps:

1. Set difficulty to **simple**. As the “attacker,” visit the login page without logging in and note the session cookie’s value — this is a real, valid (if anonymous) session ID the server already issued you.
2. In a second browser profile (“victim”), manually set that exact same session cookie value, then log in normally as `alice`.
3. Back in the attacker’s original browser/profile (same session ID, never logged in there yourself), reload `/dashboard.php` — confirm you’re now looking at `alice`’s authenticated session, despite never entering her password.
4. Switch to **hard** tier and repeat steps 1–3 against the main login form — confirm the session ID now rotates the moment login succeeds, so the pre-planted ID from step 1 is worthless after step 2.
5. Still on **hard** tier, target the *other* login path instead: have the victim log in with “Remember me” checked (issuing a `remember_token` cookie), then simulate the victim’s browser restarting by clearing only the session cookie (not `remember_token`) and revisiting the app. Note the fresh session ID the remember-me auto-login issues you — then reason about whether an attacker who pre-planted *that* ID before this auto-login fired would have won the same way as step 3.

Success check: Full access to `alice`’s session via a pre-planted ID at simple/intermediate tier; confirmation that the *main* login path closes this at hard/expert, while the remember-me auto-login path still never rotates the ID at any tier.

Report it: High at simple/intermediate. Title: “*Session Fixation in Login Flow Allows Pre-Authentication Session Hijacking.*” Note explicitly that this is distinct from session *theft*: the attacker never needs to see a cookie value the victim generated, only to supply one of their own in advance. At hard tier, note the finding more narrowly as “*Session Fixation Persists via ‘Remember Me’ Auto-Login Path*” — a fix applied to the obvious code path (password login) but not a second path reaching the identical privilege change.

Lab 24: Clickjacking — Framing the 2FA Toggle

Tools: A local scratch HTML file with an `<iframe>` **Tier:** hard and expert **Objective:** Load a page from this app inside an attacker-controlled frame, and find the one page that's still frameable even once the rest of the app isn't.

Steps:

1. Build a minimal local HTML file: `<iframe src="http://<target>/vulnapp/dashboard.php" width="800" height="600"></iframe>` . Set difficulty to **hard**, log in as any user in the same browser, then open your local file — confirm the iframe refuses to render the page.
2. Log in as `carol` specifically (the 2FA-enabled account) and let the login flow redirect you to `/user/verify_2fa.php` . Change your iframe's `src` to that exact URL instead, reload your local file, and confirm — unlike the dashboard — it loads without restriction.
3. Extend your scratch page into a minimal clickjacking PoC: position the iframe so the real page's content sits underneath, then overlay a decoy element (e.g. a styled `<button>`) directly on top of where the real page's actionable control sits, using absolute positioning and a transparent (or near-transparent) iframe.
4. Switch difficulty to **expert** and reload the exact same framing page (still pointed at `/user/verify_2fa.php`). Confirm it now refuses to render, same as the dashboard did in step 1.

Success check: `/user/verify_2fa.php` loads inside your iframe at hard tier and is blocked at expert tier, with a working overlay-alignment demo from step 3.

Report it: Medium, upgradable to **High** if paired with a believable pretext. Title: *"Clickjacking on Standalone 2FA Verification Page via Missing X-Frame-Options."* Note in your report that this is the same root cause as the Lab 14/19 "forgotten endpoint" pattern — a protection applied to the shared page template, with one standalone page built outside it and never re-checked individually.

Lab 25: Cache Poisoning — Poisoning the Login Page for Everyone

Tools: curl, run as two genuinely separate requests (“attacker” and “victim”) **Tier:** simple through expert **Objective:** Make the login page serve YOUR injected content to a completely different, later visitor who never sent anything unusual at all.

Steps:

1. Set difficulty to **simple**. Request `/index.php` normally with curl and view the page source — note the `<link rel="canonical">` tag's value.
2. As the “attacker,” resend the exact same request, this time adding a header: `curl -H "X-Forwarded-Host: evil.example" http://<target>/vulnapp/`. Confirm the canonical link now contains `evil.example`.
3. As the “victim,” send a completely clean request to that exact same URL — no special headers at all: `curl http://<target>/vulnapp/`. Confirm the victim's response *still* shows `evil.example` — this is the actual poisoning: your one crafted request changed what a different, later, header-free request receives.
4. Switch to **intermediate**. Repeat steps 2–3, but add a cache-busting query parameter to both requests (e.g. `?cb=1`) to force a fresh cache entry for this specific test — confirm the same poisoning still works once you know to bust the cache per test.
5. Switch to **hard**. Confirm the injected value is now HTML-escaped in the page source (no raw markup breakout) — but note it's still used, unescaped as a *destination*, in the canonical link's actual URL, making this an open-redirect-flavored poisoning instead.
6. Switch to **expert**. Confirm `X-Forwarded-Host` no longer affects the canonical link at all — then repeat the same attacker/victim pattern from steps 2–3 using `Accept-Language` instead, targeting the page's “Preferred language” banner. Confirm that *second*, unrelated header is still poisoning the same cached page.

Success check: A documented attacker request + a separate, clean victim request to the identical URL, with the victim's response showing

the attacker's injected value, at every tier through to expert's residual `Accept-Language` gap.

Report it: High at simple/intermediate/hard. Title: *"Cache Poisoning via Unkeyed X-Forwarded-Host Header."* The key point for your write-up: a cache that doesn't vary by a header it reflects turns a single malicious request into an attack on every subsequent visitor, not just the attacker's own session. At expert, a **Low-Medium** follow-on finding on the `Accept-Language` gap — same root cause, lower-impact banner, worth reporting as a second instance of the same unfixed pattern.

Lab 26: DOM XSS — The Search Deep Link That Never Touches the Server

Tools: Browser address bar only — no Burp, no server-side requests of any kind **Tier:** simple through expert **Objective:** Get JavaScript to execute from a URL fragment that the server never even sees, and understand why none of this app's server-side XSS protections apply to it at all.

Steps:

1. Set difficulty to **simple**. Log in as any user, navigate to `/user/tickets.php`, then edit the URL to append `#q=<script>alert(1)</script>` and reload. Confirm the alert fires, then open DevTools' Network tab, repeat the reload, and confirm this exact payload never appears in any outgoing request — the fragment (`#...`) is a browser-only construct, never transmitted to the server.
2. Switch to **intermediate**. Confirm the same `#q=<script>...` payload is now neutralized. Without using the word "script" anywhere, construct a payload using a different tag's event-handler attribute instead (e.g. `#q=<img src=x onerror=alert(1)>`) and confirm it fires.
3. Switch to **hard**. Confirm your step-2 payload no longer works (a naive allowlist now strips most tags). Construct an `<a href>` tag that includes a legitimate `href` attribute *and* a separate event-handler attribute on the same tag, and confirm the whole tag — handler included — survives the filter.

4. Switch to **expert**. Confirm every payload you've built in this lab so far now renders as inert, literal text instead of executing.

Success check: Confirmed execution at simple/intermediate/hard via three different payload shapes, each demonstrated to never appear in any server-bound request, and confirmed closure at expert.

Report it: Medium-High at simple through hard. Title: *"DOM-Based XSS via Unsanitized URL Fragment in Search Deep Link."* The single most important line in this report is explaining *why* this is a distinct finding from the Lab 7/11 server-side stored XSS, even though the payloads look similar: there is no server-side fix that touches this bug at all, since the vulnerable code path (reading `location.hash`, writing via `innerHTML`) runs entirely in the victim's own browser.

Bonus Phase — Server-Side Request, XML & Workflow Flaws (Labs 27–30)

These four labs cover the SSRF, XXE, 2FA bypass, and expanded business-logic modules added alongside the previous bonus phase. Unlike that phase, these all involve server-side request handling or multi-step state, so expect each lab to take a bit longer per tier.

Lab 27: SSRF — Abusing the Ticket Link Preview

Tools: curl or Burp Suite; for the expert-tier step, a server you control that can respond with an HTTP redirect (a free redirect-hosting service, or a one-line script on any machine you control, works fine) **Tier:** simple through expert **Objective:** Get the server itself to fetch a resource it was never supposed to reach, via a "paste a URL, get a preview" feature on ticket composition.

Steps:

1. Set difficulty to **simple**. Log in, open ticket composition, and submit `file:///etc/passwd` (or an app file like `includes/db.php`) as the link to preview. Confirm the file's contents come back in the preview.
2. Switch to **intermediate**. Confirm `file://` URLs are now rejected, and confirm a direct `http://127.0.0.1/` or `http://`

`localhost/` is also blocked. Instead, submit `http://127.1/` (or octal `http://0177.0.0.1/`) and confirm the preview still reaches loopback — same address, different spelling, outside the blacklist's exact string matches.

3. Switch to **hard**. Confirm both `127.1` and `0177.0.0.1` are now correctly blocked. Instead, host a URL that responds with an HTTP redirect (3xx) to `http://127.0.0.1/` somewhere, and submit *that* URL to the preview feature — confirm the preview follows the redirect and reaches the internal address anyway, since only the original, externally-hosted URL was validated.
4. Switch to **expert**. Confirm the same redirect trick from step 3 no longer works. Instead, submit `http://169.254.169.254/` directly and confirm it's still fetched successfully — this address is never included in the private-range blacklist, the single most common real-world gap in exactly this kind of hand-rolled check.

Success check: Confirmed internal/local-file access at all four tiers via four distinct techniques, with the expert-tier `169.254.169.254` result documented even though nothing meaningful is actually listening there in this lab environment — the point is proving the address is *reachable*, which on a real cloud deployment is exactly how SSRF escalates to credential theft.

Report it: **Critical** at simple (arbitrary local file read via `file://`), **High** at intermediate/hard (internal network access via blacklist/redirect bypass), **Medium** at expert (missing cloud-metadata range).

Lab 28: XXE — Bulk Ticket Import

Tools: Burp Suite or curl, a scratch XML file **Tier:** simple and intermediate (conceptual beyond that — see step 4) **Objective:** Get the server's XML parser to read a local file via a crafted `<!DOCTYPE>` , then recognize why a “fix” that merely changes parser APIs can still be fully exploitable.

Steps:

1. Set difficulty to **simple**. As `admin` , open Bulk Import Tickets and submit:

```

<?xml version="1.0"?>
<!DOCTYPE tickets [<!ENTITY xxe SYSTEM "file:///etc/
passwd">]>
<tickets><ticket><subject>test</
subject><message>&xxe;</message></ticket></tickets>

```

Confirm the imported ticket's message contains the file's contents.

2. Switch to **intermediate**. Resend the *exact same* payload unchanged. Confirm it still works — the fix switched to `DOMDocument` with `LIBXML_NOENT`, a flag that sounds protective but actually means “substitute entity references,” the opposite of blocking them.
3. Switch to **hard**. Resend the same payload again and confirm it now fails outright (any `<!DOCTYPE` is rejected before parsing) — this is the actual fix, not the intermediate tier's parser swap.
4. Without running anything live: read the Bulk Import Tickets challenge write-up and README's description of (a) blind/out-of-band XXE via a parameter entity referencing an attacker-hosted external DTD, and (b) entity-expansion (“billion laughs”) denial of service. Write one paragraph on each, as if explaining the risk to a developer who fixed the DOCTYPE-rejection bug and now believes XXE is fully closed.

Success check: Confirmed working exfiltration at simple AND intermediate tier with the identical payload (proving the “fix” didn't fix anything), confirmed rejection at hard tier, and two written paragraphs for step 4.

Report it: Critical for simple/intermediate — and make the intermediate-tier report explicit that it's the *same* vulnerable condition as simple, just reached through code that looks different. Title: *“LIBXML_NOENT Misconception Leaves XXE Fully Exploitable Despite Parser Change.”*

Lab 29: 2FA Bypass — Breaking TOTP Step-Up Auth

Tools: Browser or curl; a TOTP code generator for carol's seeded secret `JBSWY3DPEHPK3PXP` — `oathtool --totp -b`

JBSWY3DPEHPK3PXP on the CLI, or enter that secret manually into any standard authenticator app **Tier:** simple through expert, carol's account only **Objective:** Confirm the second factor genuinely works when followed correctly, then find four separate ways around it.

Steps:

1. Set difficulty to **hard** (any tier where codes are actually checked works for this baseline step). Log in as carol, generate a real code with `oathtool --totp -b JBSWY3DPEHPK3PXP`, submit it, and confirm you land on the dashboard normally — establishing the control is real before trying to bypass it.
2. Switch to **simple**. Log in as carol again and this time, *without* submitting any code at all, browse directly to `/dashboard.php`. Confirm full access anyway.
3. Switch to **intermediate**. Confirm `/dashboard.php` now correctly bounces an unverified session back to the code prompt. Instead, with that same still-unverified session, request `/api/tickets.php` directly — confirm it returns ticket data anyway.
4. Switch to **hard**. Confirm that API gap is now closed. Instead, submit a long sequence of incorrect 6-digit codes at the code prompt in a tight loop and confirm none of them trigger any lockout or delay at all.
5. Switch to **expert**. Confirm repeated bad guesses now correctly rate-limit. Log in once, enter a real code, and check “Trust this device” — note the resulting cookie's name and value. Clear your session (but not that cookie), and reason about how that cookie's value relates to carol's username alone — then explain why that makes it forgeable for any account without ever producing a valid code.

Success check: Documented proof of all four tiered bugs — decorative verification (simple), a forgotten endpoint (intermediate), no rate limiting (hard), and a weak/forgeable trusted-device token (expert) — plus the working baseline from step 1 proving the control isn't a prop.

Report it: Simple is **Critical** (“*Second Factor Never Actually Enforced Before Granting Full Session*”). Intermediate is **High** (“*Partial-Auth Session Bypasses 2FA on Unmigrated API Endpoint*”). Hard is **Medium-High**, same absence-of-control category as the login-brute-

force findings earlier in this manual. Expert is **High** (*"Forgeable 'Trust This Device' Cookie Permanently Bypasses 2FA"*).

Lab 30: Business Logic — Breaking the Escalation Approval Workflow

Tools: Browser, Burp Suite (to send a request a UI control wouldn't normally allow) **Tier:** simple through expert **Objective:** A regular user can request their ticket be escalated to Urgent; only staff should be able to approve that. Find every tier's gap in that approval gate.

Steps:

1. Set difficulty to **simple**. Log in as a regular user (`alice` or `bob`), submit a ticket, and click "Request Urgent" on it. Confirm the priority jumps to `urgent` immediately — no approval step exists at all despite the UI implying one.
2. Switch to **intermediate**. Click the button once and confirm it visually disables afterward — then use Burp to resend the exact same POST request directly. Confirm the escalation still applies immediately regardless of the disabled button.
3. Switch to **hard**. Confirm a genuine server-side pending state now exists and the request step correctly enforces it. Staying logged in as the same non-staff requester, locate the staff approval endpoint the Support Queue UI posts to, and `POST` to it yourself. Confirm you can approve (or deny) your own pending request, despite never being `support` or `admin`.
4. Switch to **expert**. Confirm that self-approval gap is now closed. Instead, get a staff account (`sam`) to deny your request, then immediately resubmit the identical escalation request on the same ticket, repeatedly, in quick succession. Confirm there's no cooldown or cap at all on how many times you can re-request after a denial.

Success check: Confirmed instant unauthorized escalation at simple/intermediate (two different mechanisms), confirmed unauthorized self-approval at hard, and confirmed unlimited re-request flooding at expert.

Report it: Simple/intermediate are each their own **High** finding — note explicitly for intermediate that *"client-side state visually implying an approval step is not the same as a server-side approval step existing."*

Hard is **Critical** (*“Missing Role Check on Escalation-Approval Endpoint Allows Self-Approval”*). Expert is **Low-Medium**, same absence-of-a-limit category as this manual’s other “missing control, not a broken one” findings.

After Lab 30

You’ve now run the full workflow this app was built to teach: recon, mapping, straightforward exploitation, filter evasion, chained/secondary flaws, logic bugs, API testing, JWT auth flaws, session/client-side security, server-side request forgery, XML parsing flaws, step-up authentication bypasses, a full business-logic workflow — and, critically, turning all of it into a report a real program would actually accept. From here, the honest next step (per this app’s own README) is DVWA and PortSwigger’s Web Security Academy for the vulnerability categories a single custom app still can’t cover (GraphQL, HTTP smuggling, SSTI, prototype pollution, and more) — you now have the workflow habits to get real value out of them faster.